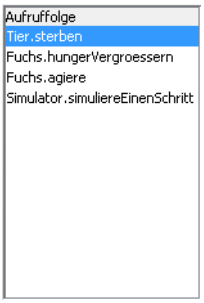
Der Bereich *Lokale Variablen* zeigt die Werte der lokalen Variablen und Parameter der gerade ausgeführten Methode bzw. des gerade ausgeführten Konstruktors. Lokale Variablen erscheinen in diesem Bereich erst dann, wenn sie initialisiert wurden, da sie erst zu diesem Zeitpunkt innerhalb der virtuellen Maschine sichtbar werden.

Jede Variable in diesen Bereichen, die auf ein Objekt verweist, kann mit einem Doppelklick weiter untersucht werden.

F.4 Die Anzeige der Aufruffolge

Abbildung F.5 zeigt den Fensterausschnitt für die Anzeige der Aufruffolge, hier mit einer Sequenztiefe von vier Methoden. Die Methoden erscheinen im Format `Klasse.methode` in der Sequenz, unabhängig davon, ob es sich um eine Klassen- oder Instanzmethode handelt. Konstruktoren erscheinen als `Klasse.<init>` in der Sequenz.

Abbildung F.5
Eine Aufruffolge.



Die Aufruffolge arbeitet nach dem Stapelprinzip, bei dem die Methode ganz oben diejenige bezeichnet, in der gerade der Kontrollfluss hält. Die Variablen-Anzeigen zeigen jeweils Details zu der Methode oder dem Konstruktor, der aktuell in der Aufruffolge hervorgehoben ist. Das Selektieren einer anderen Zeile aus der Folge bewirkt, dass die Inhalte der anderen Anzeige-Bereiche aktualisiert werden.

F.5 Die Thread-Anzeige

Die Thread-Anzeige geht über den Rahmen dieses Buches hinaus und wird hier nicht weiter behandelt.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:04 Uhr

ANHANG

G

Testwerkzeuge für Modultests mit JUnit

In diesem Anhang geben wir einen kurzen Überblick über die wichtigsten Eigenschaften der BlueJ-Unterstützung für Modultests im JUnit-Stil. Weitere Einzelheiten können im Test-Tutorial nachgelesen werden, das auf der BlueJ-Website verfügbar ist.

G.1 Aktivieren der Test-Funktionalität

Um die Test-Funktionalität von BlueJ zu aktivieren, müssen Sie sicherstellen, dass unter WERKZEUGE/EINSTELLUNGEN/DIVERSES die Box *Testwerkzeuge anzeigen* selektiert ist. Das Hauptfenster von BlueJ enthält dann einige weitere Knöpfe, die bei einem geöffneten Projekt aktiviert sind.

G.2 Eine Testklasse erzeugen

Sie erzeugen eine Testklasse, indem Sie einen Rechtsklick auf einer Klasse im Klassendiagramm ausführen und im Kontextmenü TESTKLASSE ERZEUGEN auswählen. Der Name der Testklasse wird automatisch festgelegt, indem Test als Suffix für den Namen der zugeordneten Klasse verwendet wird. Alternativ können Sie eine Testklasse auch über den Knopf *neue Klasse…* erzeugen, indem Sie als Klassentyp *Unit-Test* angeben. Dann können Sie den Namen der Klasse frei festlegen.

Testklassen werden mit `<<unit test>>` im Klassendiagramm annotiert und haben eine andere Farbe als gewöhnliche Klassen.

G.3 Eine Testmethode erzeugen

Testmethoden können interaktiv erzeugt werden. Eine Sequenz von Benutzerinteraktionen mit dem Klassendiagramm und der Objektleiste werden aufgezeichnet und als eine Folge von Java-Anweisungen und Deklarationen in einer Methode der Testklasse festgehalten. Sie starten eine Aufzeichnung, indem Sie ERZEUGE TESTMETHODE aus dem Kontextmenü einer Testklasse selektieren. Sie werden dann nach dem Namen der neuen Testmethode gefragt. Frühere JUnit-Versionen (bis Version 3) setzten voraus, dass dieser Name mit *test* begann. Dies ist heute nicht mehr der Fall. Das *Aufzeichnung*-Symbol links vom Klassendiagramm erscheint nun rot und die Knöpfe *Beenden* und *Abbrechen* werden verfügbar.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:04 Uhr

Sobald eine Aufzeichnung gestartet wurde, werden alle Objekterzeugungen oder Methodenaufrufe Teil des Codes der neu erzeugten Methode. Selektieren Sie *Beenden*, um die Aufzeichnung zu speichern, oder *Abbrechen*, um die Aufzeichnung abzubrechen und die Testklasse unverändert zu lassen.

Testmethoden werden im Quelltext der Testklassen mit der Annotation `@Test` versehen.

G.4 Zusicherungen bei Tests

Während der Aufzeichnung einer Testmethode öffnet jeder Methodenaufruf mit einem Ergebnis das Fenster *Methodenergebnis*. In diesem wird die Möglichkeit gegeben, eine Zusicherung zu dem Ergebniswert zu formulieren, indem die Box *Sichere zu, dass* selektiert wird. Das zugeordnete Menü enthält eine Menge von möglichen Zusicherungen zu dem Testergebnis. Wenn eine Zusicherung gemacht wurde, wird diese in der Testmethode durch entsprechenden Code umgesetzt, der zu einem `AssertionError` führt, falls die Zusicherung nicht zutrifft.

G.5 Tests ausführen

Individuelle Testmethoden können ausgeführt werden, indem sie aus dem Kontextmenü der zugehörigen Testklasse ausgewählt werden. Ein erfolgreicher Test wird durch eine Meldung in der Statuszeile des Hauptfensters angezeigt. Ein nicht erfolgreicher Test lässt das Fenster *Testergebnisse* erscheinen. Selektieren von ALLES TESTEN im Kontextmenü der Testklasse lässt alle Tests dieser Klasse ausführen. Das Fenster *Testergebnisse* liefert dann Informationen über den Erfolg oder Misserfolg jeder einzelnen Methode.

G.6 Testgerüste

Der Inhalt der Objektleiste kann als Testgerüst festgehalten werden, indem OBJEKTZUSTAND SPEICHERN aus dem Kontextmenü einer Testklasse aufgerufen wird. Das Erzeugen eines Testgerüstes bewirkt, dass in der Testklasse für jedes Objekt eine Instanzvariable angelegt wird und Anweisungen in die Methode `setUp` eingefügt werden, die den exakten Status der Objekte wie auf der Objektleiste herstellen. Die Objekte werden dadurch von der Objektleiste entfernt.

Die Methode `setUp` wird in der Testklasse mit `@Before` annotiert und automatisch vor jeder Ausführung einer Testmethode ausgeführt, sodass alle Objekte eines Testgerüsts für alle Tests zur Verfügung stehen.

Die Objekte eines Testgerüsts können wieder auf der Objektleiste erzeugt werden, indem TESTGERÜST WIEDERHERSTELLEN aus dem Kontextmenü der Testklasse gewählt wird.

Die Methode `tearDown` (mit der Annotation `@After`) wird nach jeder Testmethode aufgerufen. Sie kann dazu verwendet werden, um eventuelle Aufräumarbeiten nach dem Testen – sofern benötigt – durchzuführen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:04 Uhr

ANHANG

H Werkzeuge für die Teamarbeit

In diesem Abschnitt werden wir kurz auf die Werkzeuge eingehen, die Ihnen für die Teamarbeit zur Verfügung stehen.

Zum Umfang von BlueJ gehören auch verschiedene Werkzeuge für die Arbeit im Team, die allesamt auf einem Quelltextarchiv-Modell basieren. Bei diesem Modell wird ein Archiv-Server eingerichtet, der über das Internet für alle Teammitglieder zugänglich ist.

Der Server sollte von einem Administrator eingerichtet werden. BlueJ unterstützt sowohl Subversion- als auch CVS-Archive.

H.1 Server-Einrichtung

Die Einrichtung des Archiv-Servers sollte einem erfahrenen Administrator übertragen werden. Ausführliche Anweisungen hierzu finden Sie im Dokument „BlueJ Teamwork Repository Configuration“, auf das Sie über BLUEJ-TUTORIAL im HILFE-Menü von BlueJ zugreifen können.

H.2 Teamarbeit-Funktionalität aktivieren

Die Teamarbeitswerkzeuge von BlueJ sind zu Beginn verborgen. Um die Werkzeuge anzuzeigen, öffnen Sie das Dialogfeld WERKZEUGE/EINSTELLUNGEN und aktivieren Sie auf der Registerseite DIVERSES die Option *Teamarbeitswerkzeuge anzeigen*. Damit blenden Sie drei zusätzliche Schalter (*Aktualisieren*, *Abgeben*, *Status*) ein und legen im Menü WERKZEUGE ein Untermenü namens TEAMARBEIT an.

H.3 Ein Projekt zur gemeinsamen Nutzung einrichten

Um ein gemeinsames Projekt einzurichten, erzeugt eines der Teammitglieder das Projekt als normales BlueJ-Projekt. Mit der Funktion *Projekt gemeinsam nutzen* im Menü WERKZEUGE/TEAMARBEIT kann er das Projekt dann für alle zugänglich machen. Mit dieser Funktion wird eine Kopie des Projekts in das zentrale Archiv abgelegt. Der Servername und Zugriffsdetails müssen in einem Dialogfeld angegeben wer-

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:04 Uhr